

Universidad Tecnológica Nacional



Tecnicatura Universitaria en Programación a Distancia

Trabajo Práctico Integrador – Bases de Datos I

Food Store – Sistema de Gestión de Pedidos

Alumna: Martínez, Victoria Abigail (vickymiley30@gmail.com)

Comisión 8

Profesor: Carcaño, Nicolás

Fecha: 23 de junio del 2026

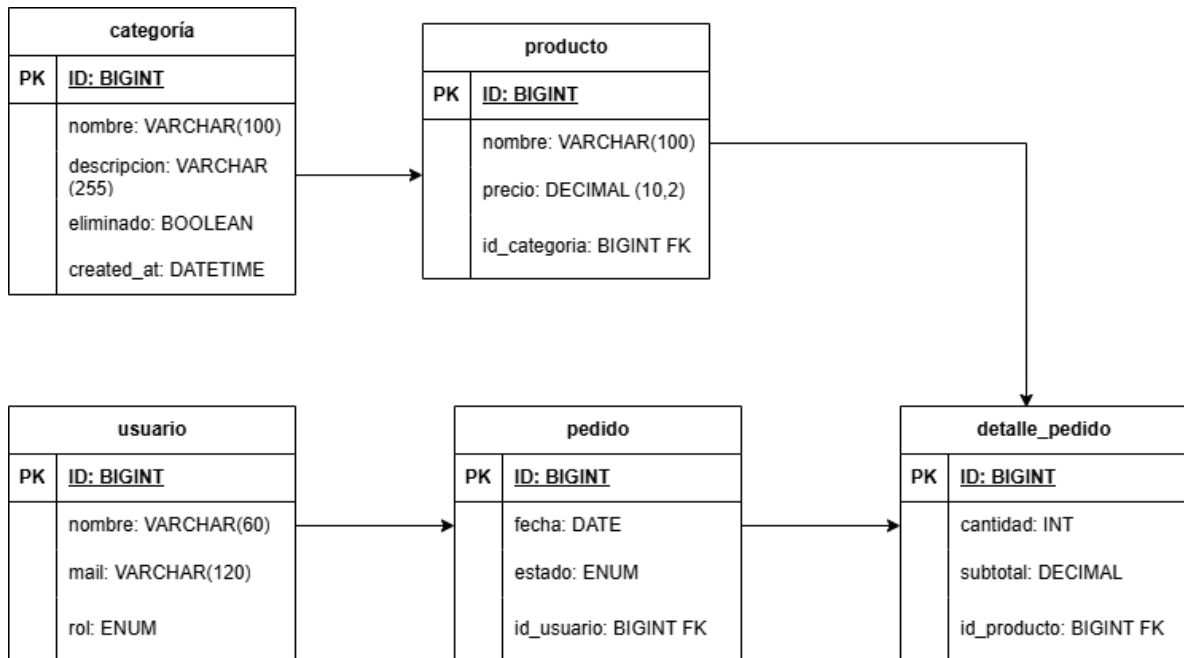
Resumen Ejecutivo

El presente Trabajo Final Integrador tiene como objetivo implementar una base de datos relacional para el sistema Food Store, un gestor de pedidos de comida desarrollado en la materia Programación II. Se diseñó el modelo entidad-relación, se implementaron las tablas con sus constraints (PK, FK, UNIQUE, CHECK), se realizó una carga masiva de más de 15000 registros, se desarrollaron consultas avanzadas con JOIN, GROUP BY y subconsultas, se aplicaron medidas de seguridad mediante usuarios con privilegios mínimos y procedimientos almacenados, y se trabajaron transacciones y niveles de aislamiento para el control de concurrencia.

Reglas de negocio

- Un producto debe pertenecer obligatoriamente a una categoría.
- El precio de un producto no puede ser negativo (CHECK precio ≥ 0).
- El stock de un producto no puede ser negativo (CHECK stock ≥ 0).
- El mail de cada usuario debe ser único (UNIQUE mail).
- Un pedido debe tener obligatoriamente un usuario asociado.
- La cantidad en detalle_pedido debe ser mayor a 0 (CHECK cantidad > 0).
- Todas las entidades implementan eliminación lógica (soft delete) mediante el campo `eliminado = TRUE`, sin borrar físicamente los registros.

DER / Modelo Relacional



Decisiones de Diseño y Constraints

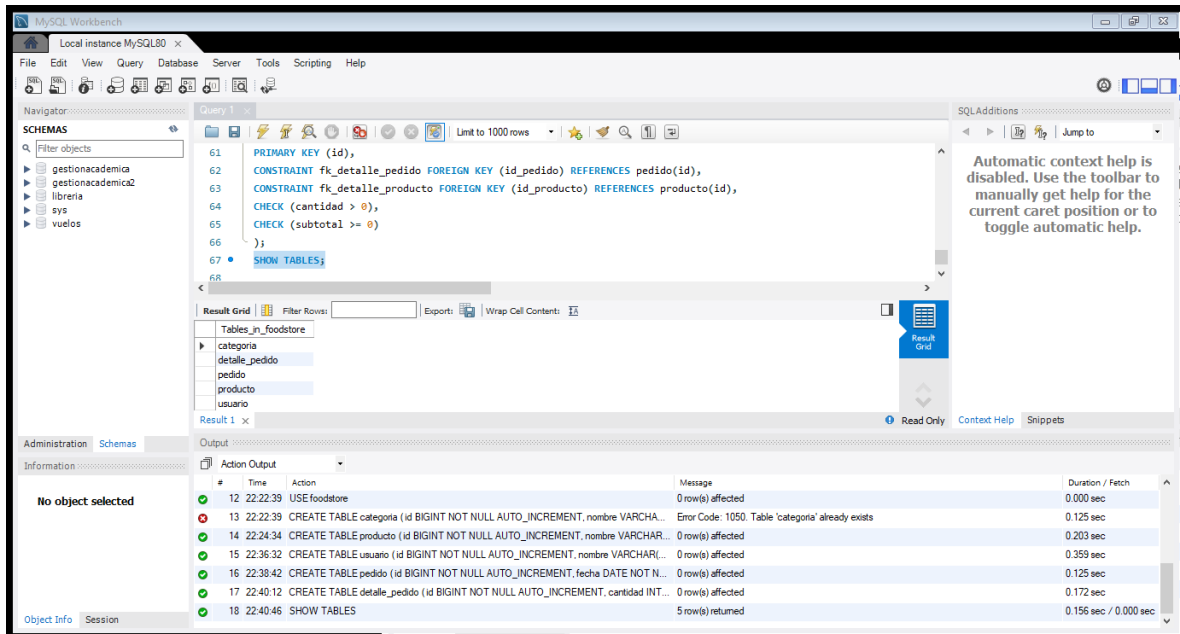
El modelo relación fue diseñado siguiendo el sistema Food Store desarrollado en Programación II. Las principales decisiones tomadas fueron:

- **Soft delete:** todas las tablas tienen el campo `eliminado` `BOOLEAN` `DEFAULT FALSE` para no borrar registros físicamente y mantener el historial.
- **AUTO_INCREMENT:** el campo `id` de cada tabla se genera automáticamente.
- **UNIQUE:** el campo `mail` de la tabla `usuario` y el campo `nombre` de `categoría` son únicos.
- **CHECK:** el `precio` y el `stock` de `producto` no pueden ser negativos. La cantidad en `detalle_pedido` debe ser mayor a 0.
- **FOREIGN KEYS:** se definieron FK entre `producto` y `categoría`, `pedido` y `usuario`, `detalle_pedido` con `pedido` y `producto`.
- **ENUM:** los campos `rol`, `estado` y `forma_pago` usan `ENUM` para restringir los valores posibles.

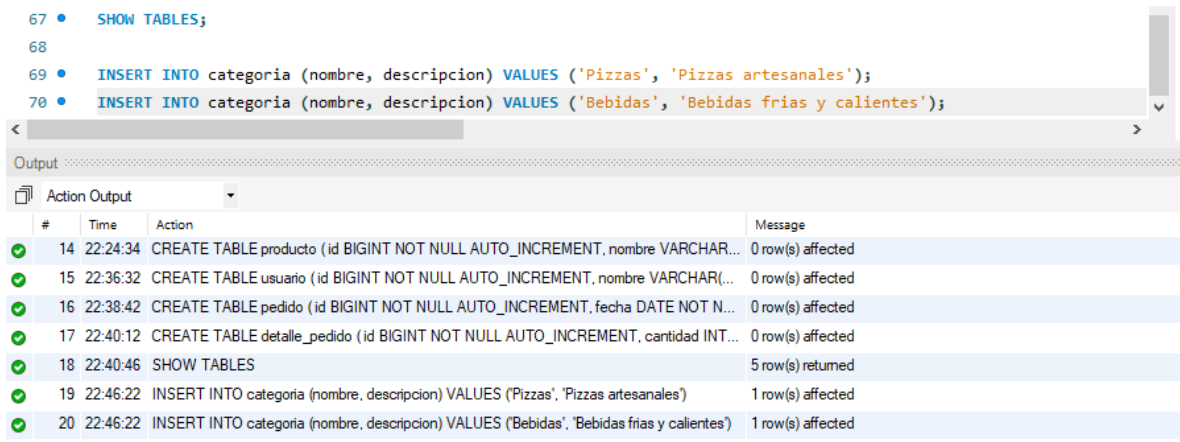
Evidencias

Etapas 1 – Modelado y Constraints:

Capturas de las tablas creadas (SHOW TABLES)



Captura de las inserciones correctas:



Captura de las 2 inserciones incorrectas para mostrar que los constraints funcionan:

Ejemplo: nombre duplicado y precio negativo.

72	•	INSERT INTO categoria (nombre, descripcion) VALUES ('Pizzas', 'Otra descripcion');	
Output			
Action Output			
#	Time	Action	Message
✓ 15	22:36:32	CREATE TABLE usuario (id BIGINT NOT NULL AUTO_INCREMENT, nombre VARCHAR(...	0 row(s) affected
✓ 16	22:38:42	CREATE TABLE pedido (id BIGINT NOT NULL AUTO_INCREMENT, fecha DATE NOT N...	0 row(s) affected
✓ 17	22:40:12	CREATE TABLE detalle_pedido (id BIGINT NOT NULL AUTO_INCREMENT, cantidad INT...	0 row(s) affected
✓ 18	22:40:46	SHOW TABLES	5 row(s) returned
✓ 19	22:46:22	INSERT INTO categoria (nombre, descripcion) VALUES ('Pizzas', 'Pizzas artesanales')	1 row(s) affected
✓ 20	22:46:22	INSERT INTO categoria (nombre, descripcion) VALUES ('Bebidas', 'Bebidas frías y calientes')	1 row(s) affected
✗ 21	22:48:45	INSERT INTO categoria (nombre, descripcion) VALUES ('Pizzas', 'Otra descripcion')	Error Code: 1062. Duplicate entry 'Pizzas' for key 'categoria.nombre'

73	•	INSERT INTO producto (nombre, precio, stock, id_categoria) VALUES ('Pizza', -100, 10, 1);	
Output			
Action Output			
#	Time	Action	Message
✓ 16	22:38:42	CREATE TABLE pedido (id BIGINT NOT NULL AUTO_INCREMENT, fecha DATE NOT N...	0 row(s) affected
✓ 17	22:40:12	CREATE TABLE detalle_pedido (id BIGINT NOT NULL AUTO_INCREMENT, cantidad INT...	0 row(s) affected
✓ 18	22:40:46	SHOW TABLES	5 row(s) returned
✓ 19	22:46:22	INSERT INTO categoria (nombre, descripcion) VALUES ('Pizzas', 'Pizzas artesanales')	1 row(s) affected
✓ 20	22:46:22	INSERT INTO categoria (nombre, descripcion) VALUES ('Bebidas', 'Bebidas frías y calientes')	1 row(s) affected
✗ 21	22:48:45	INSERT INTO categoria (nombre, descripcion) VALUES ('Pizzas', 'Otra descripcion')	Error Code: 1062. Duplicate entry 'Pizzas' for key 'categoria.nombre'
✗ 22	22:49:45	INSERT INTO producto (nombre, precio, stock, id_categoria) VALUES ('Pizza', -100, 10, 1)	Error Code: 3819. Check constraint 'producto_chk_1' is violated.

Etap 2 – Carga Masiva

Cantidad de pedidos generados (5.000 registros).

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

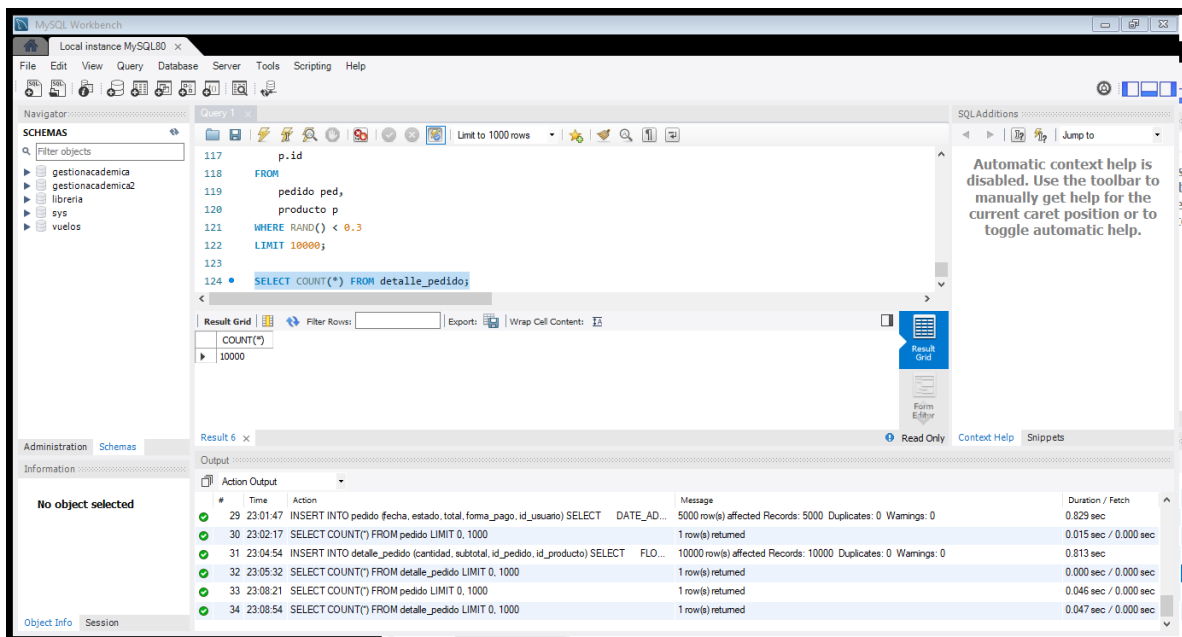
```

105 FROM
106 information_schema.columns a,
107 information_schema.columns b
108 LIMIT 5000;
109
110 SELECT COUNT(*) FROM pedidos;
111
112 INSERT INTO detalle_pedido (cantidad, subtotal, id_pedido, id_producto)
  
```

The output pane shows the results of the query, including the execution of the INSERT statement and the resulting 5000 records in the detalle_pedido table. The output also shows the execution of the SELECT COUNT(*) query, which returns 5000.

#	Time	Action	Message	Duration / Fetch
✓ 28	23:00:59	INSERT INTO producto (nombre, precio, descripcion, stock, imagen, disponible, id_categoria, ...)	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.031 sec
✓ 29	23:01:47	INSERT INTO pedido (fecha, estado, total, forma_pago, id_usuario) SELECT DATE_AD...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.829 sec
✓ 30	23:02:17	SELECT COUNT(*) FROM pedido LIMIT 0, 1000	1 row(s) returned	0.015 sec / 0.000 sec
✓ 31	23:04:54	INSERT INTO detalle_pedido (cantidad, subtotal, id_pedido, id_producto) SELECT FLO...	10000 row(s) affected Records: 10000 Duplicates: 0 Warnings: 0	0.813 sec
✓ 32	23:05:32	SELECT COUNT(*) FROM detalle_pedido LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
✓ 33	23:08:21	SELECT COUNT(*) FROM pedido LIMIT 0, 1000	1 row(s) returned	0.046 sec / 0.000 sec

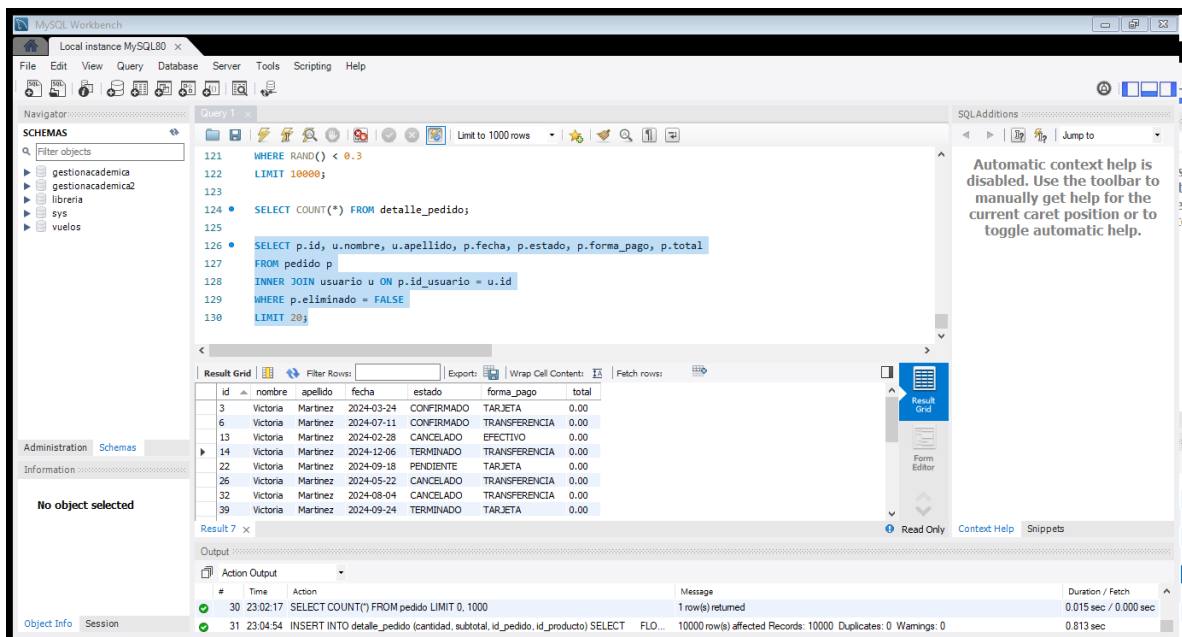
Cantidad de detalles de pedido generados (10.000 registros).



Total de registros: 15.000+ registros distribuidos en las tablas principales.

Etap 3 – Consultas avanzadas

Consulta 1 - JOIN: Mostrar pedidos con el nombre del usuario.



Consulta 2 - JOIN: Mostrar productos con su categoría.

The screenshot shows MySQL Workbench with two queries. The first query (lines 126-130) joins the 'pedido' table with the 'usuario' table on 'p.id_usuario = u.id', filtering for 'p.eliminado = FALSE' and limiting to 20 rows. The second query (lines 132-135) joins the 'producto' table with the 'categoria' table on 'p.id_categoria = c.id', filtering for 'p.eliminado = FALSE'.

Query 1 Results:

id	nombre	precio	stock	categoria
9	Hamburguesa Clasica	10000.00	50	Hamburguesas
10	Hamburguesa Doble	15000.00	40	Hamburguesas
11	Pizza Comun	8000.00	30	Pizzas
12	Pizza Napolitana	9000.00	30	Pizzas
13	Coca Cola	2000.00	100	Bebidas
14	Agua Mineral	1000.00	100	Bebidas
15	Tiramisu	5000.00	20	Postres
16	Brownie	3000.00	25	Postres

Query 2 Results:

#	Time	Action	Message	Duration / Fetch
35	23:11:37	SELECT p.id, u.nombre, u.apellido, p.fecha, p.estado, p.forma_pago, p.total FROM pedido p...	20 row(s) returned	0.032 sec / 0.000 sec
36	23:39:18	SELECT p.id, p.nombre, p.precio, p.stock, c.nombre AS categoria FROM producto p INNER...	8 row(s) returned	0.047 sec / 0.000 sec

Consulta 3 - GROUP BY + HAVING: Mostrar usuarios con más de 100 pedidos.

The screenshot shows MySQL Workbench with two queries. The first query (lines 132-135) joins the 'producto' table with the 'categoria' table on 'p.id_categoria = c.id', filtering for 'p.eliminado = FALSE'. The second query (lines 137-141) groups users by 'u.id, u.nombre, u.apellido' and counts the number of orders ('COUNT(p.id) AS cantidad_pedidos'), filtering for 'HAVING COUNT(p.id) > 100'.

Query 1 Results:

nombre	apellido	cantidad_pedidos
Victoria	Martinez	1007
Renzo	Martinez	997
Gloria	Martinez	975
Luis	Capello	1013
Ana	Rodriguez	1008

Query 2 Results:

#	Time	Action	Message	Duration / Fetch
36	23:39:18	SELECT p.id, p.nombre, p.precio, p.stock, c.nombre AS categoria FROM producto p INNER...	8 row(s) returned	0.047 sec / 0.000 sec
37	23:41:09	SELECT u.nombre, u.apellido, COUNT(p.id) AS cantidad_pedidos FROM usuario u INNER J...	5 row(s) returned	0.063 sec / 0.000 sec

Consulta 4 - Subconsulta: Mostrar productos con precio mayor al promedio.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with a tree view containing 'gestionacademia', 'gestionacademia2', 'libreria', 'sys', and 'vuelos'. The main editor area contains two SQL queries. The first query (lines 137-142) selects user names and surnames, counts the number of orders per user, and filters for users with more than 100 orders. The second query (lines 143-146) selects product names and prices, filtering for products with a price greater than the average price of all products and that are not deleted. The 'Result Grid' at the bottom shows the results of the second query, displaying a table with columns 'nombre' and 'precio'.

```
137 • SELECT u.nombre, u.apellido, COUNT(p.id) AS cantidad_pedidos
138 FROM usuario u
139 INNER JOIN pedido p ON u.id = p.id_usuario
140 GROUP BY u.id, u.nombre, u.apellido
141 HAVING COUNT(p.id) > 100;
142
143 • SELECT nombre, precio
144 FROM producto
145 WHERE precio > (SELECT AVG(precio) FROM producto)
146 AND eliminado = FALSE;
```

nombre	precio
Hamburguesa Clasica	10000.00
Hamburguesa Doble	15000.00
Pizza Comun	8000.00
Pizza Napolitana	9000.00

producto 10 x

Read Only Context Help Snippets

Action Output

#	Time	Action	Message	Duration / Fetch
37	23:41:09	SELECT u.nombre, u.apellido, COUNT(p.id) AS cantidad_pedidos FROM usuario u INNER JOIN...	5 row(s) returned	0.063 sec / 0.000 sec
38	23:42:26	SELECT nombre, precio FROM producto WHERE precio > (SELECT AVG(precio) FROM pro...	4 row(s) returned	0.047 sec / 0.000 sec

Captura de la Vista.

The screenshot shows the MySQL Workbench interface. The main editor area contains a SQL query (lines 144-154) that creates a view named 'vista_pedidos_usuarios' and selects the first 10 rows. The view is created based on the 'pedido' table, joined with the 'usuario' table, and filters for orders that are not deleted. The 'Result Grid' at the bottom shows the results of the view, displaying a table with columns 'id', 'nombre', 'apellido', 'fecha', 'estado', 'forma_pago', and 'total'.

```
144 FROM producto
145 WHERE precio > (SELECT AVG(precio) FROM producto)
146 AND eliminado = FALSE;
147
148 • CREATE VIEW vista_pedidos_usuarios AS
149 SELECT p.id, u.nombre, u.apellido, p.fecha, p.estado, p.forma_pago, p.total
150 FROM pedido p
151 INNER JOIN usuario u ON p.id_usuario = u.id
152 WHERE p.eliminado = FALSE;
153
154 • SELECT * FROM vista_pedidos_usuarios LIMIT 10;
```

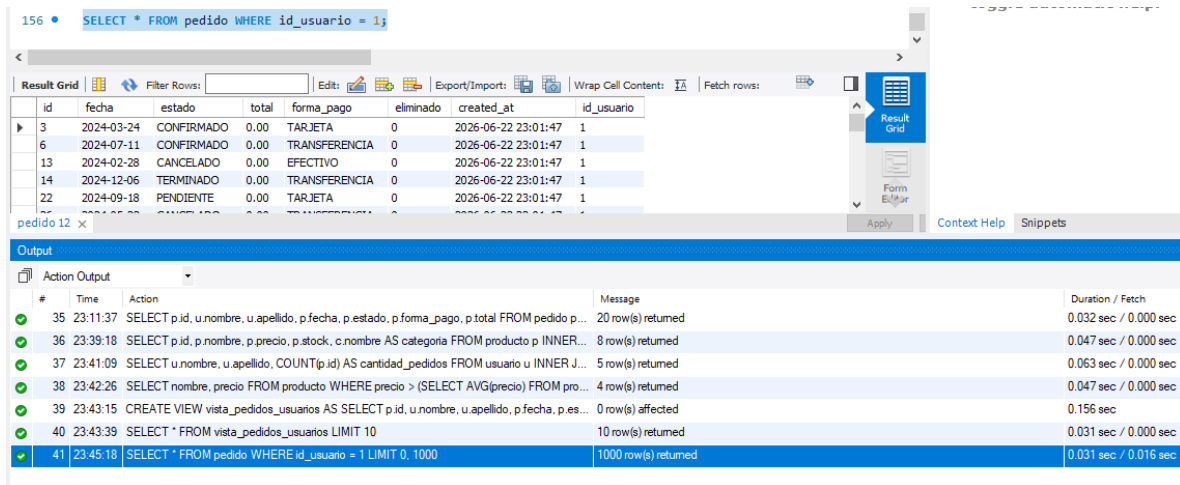
id	nombre	apellido	fecha	estado	forma_pago	total
3	Victoria	Martinez	2024-03-24	CONFIRMADO	TARJETA	0.00
6	Victoria	Martinez	2024-07-11	CONFIRMADO	TRANSFERENCIA	0.00
13	Victoria	Martinez	2024-02-28	CANCELADO	EFFECTIVO	0.00
14	Victoria	Martinez	2024-12-06	TERMINADO	TRANSFERENCIA	0.00
22	Victoria	Martinez	2024-09-18	PENDIENTE	TARJETA	0.00
26	Victoria	Martinez	2024-05-22	CANCELADO	TRANSFERENCIA	0.00
32	Victoria	Martinez	2024-08-04	CANCELADO	TRANSFERENCIA	0.00
39	Victoria	Martinez	2024-09-24	TERMINADO	TARJETA	0.00
40	Victoria	Martinez	2024-10-12	CONFIRMADO	TRANSFERENCIA	0.00
52	Victoria	Martinez	2024-08-13	PENDIENTE	TARJETA	0.00

vista_pedidos_usuarios 11 x

Read Only

Capturas de medición con/sin índice.

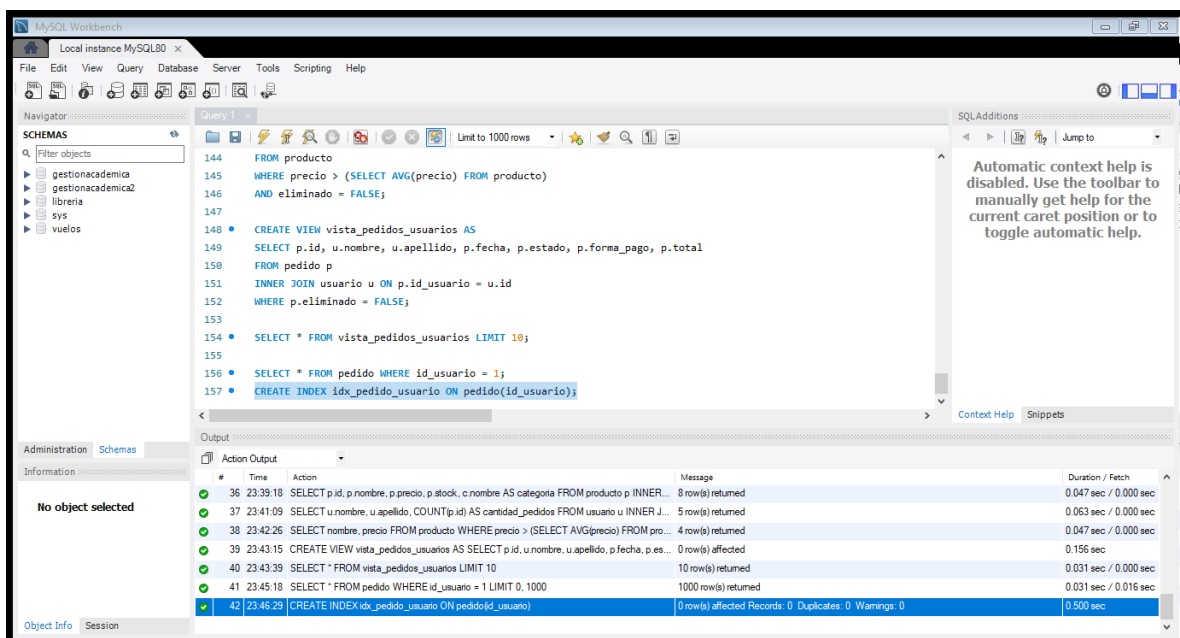
Sin índice:



156 • `SELECT * FROM pedido WHERE id_usuario = 1;`

#	Time	Action	Message	Duration / Fetch
35	23:11:37	SELECT p.id, u.nombre, u.apellido, p.fecha, p.estado, p.foma_pago, p.total FROM pedido p...	20 row(s) returned	0.032 sec / 0.000 sec
36	23:39:18	SELECT p.id, p.nombre, p.precio, p.stock, c.nombre AS categoria FROM producto p INNER...	8 row(s) returned	0.047 sec / 0.000 sec
37	23:41:09	SELECT u.nombre, u.apellido, COUNT(p.id) AS cantidad_pedidos FROM usuario u INNER J...	5 row(s) returned	0.063 sec / 0.000 sec
38	23:42:26	SELECT nombre, precio FROM producto WHERE precio > (SELECT AVG(precio) FROM pro...	4 row(s) returned	0.047 sec / 0.000 sec
39	23:43:15	CREATE VIEW vista_pedidos_usuarios AS SELECT p.id, u.nombre, u.apellido, p.fecha, p.es...	0 row(s) affected	0.156 sec
40	23:43:39	SELECT * FROM vista_pedidos_usuarios LIMIT 10	10 row(s) returned	0.031 sec / 0.000 sec
41	23:45:18	SELECT * FROM pedido WHERE id_usuario = 1 LIMIT 0. 1000	1000 row(s) returned	0.031 sec / 0.016 sec

Con índice:



MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- gestionacademica
- gestionacademica2
- libreria
- sys
- vuelos

Query 1

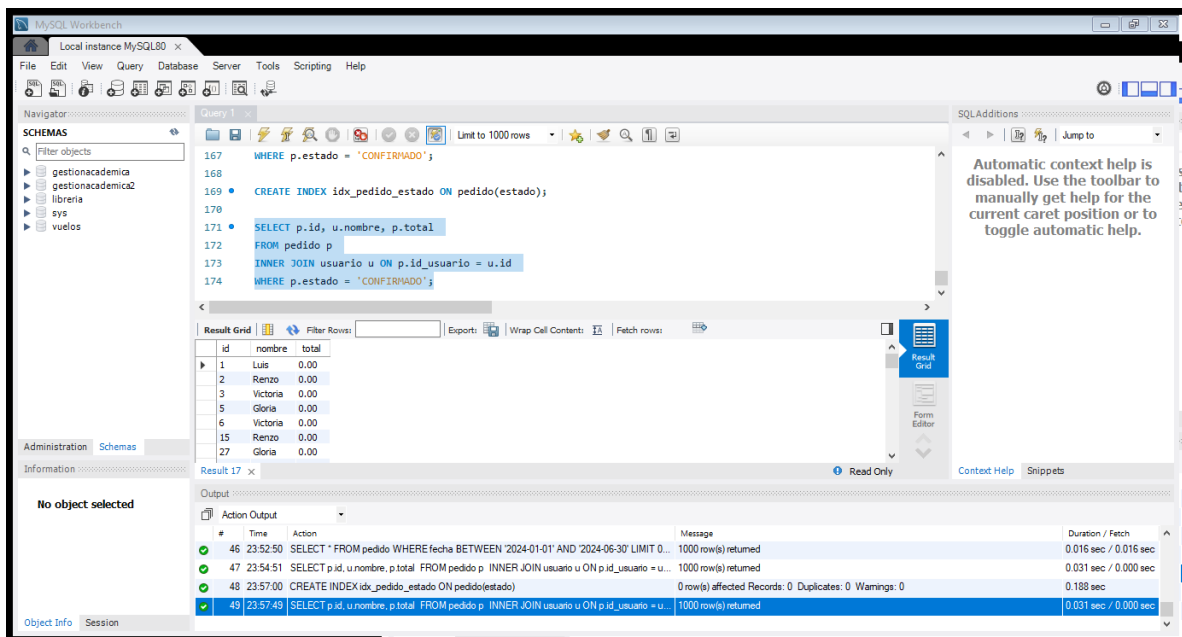
Limit to 1000 rows

```
144 FROM producto
145 WHERE precio > (SELECT AVG(precio) FROM producto)
146 AND eliminado = FALSE;
147
148 • CREATE VIEW vista_pedidos_usuarios AS
149 SELECT p.id, u.nombre, u.apellido, p.fecha, p.estado, p.foma_pago, p.total
150 FROM pedido p
151 INNER JOIN usuario u ON p.id_usuario = u.id
152 WHERE p.eliminado = FALSE;
153
154 • SELECT * FROM vista_pedidos_usuarios LIMIT 10;
155
156 • SELECT * FROM pedido WHERE id_usuario = 1;
157 • CREATE INDEX idx_pedido_usuario ON pedido(id_usuario);
```

Output

#	Time	Action	Message	Duration / Fetch
36	23:39:18	SELECT p.id, p.nombre, p.precio, p.stock, c.nombre AS categoria FROM producto p INNER...	8 row(s) returned	0.047 sec / 0.000 sec
37	23:41:09	SELECT u.nombre, u.apellido, COUNT(p.id) AS cantidad_pedidos FROM usuario u INNER J...	5 row(s) returned	0.063 sec / 0.000 sec
38	23:42:26	SELECT nombre, precio FROM producto WHERE precio > (SELECT AVG(precio) FROM pro...	4 row(s) returned	0.047 sec / 0.000 sec
39	23:43:15	CREATE VIEW vista_pedidos_usuarios AS SELECT p.id, u.nombre, u.apellido, p.fecha, p.es...	0 row(s) affected	0.156 sec
40	23:43:39	SELECT * FROM vista_pedidos_usuarios LIMIT 10	10 row(s) returned	0.031 sec / 0.000 sec
41	23:45:18	SELECT * FROM pedido WHERE id_usuario = 1 LIMIT 0. 1000	1000 row(s) returned	0.031 sec / 0.016 sec
42	23:46:29	CREATE INDEX idx_pedido_usuario ON pedido(id_usuario)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.500 sec

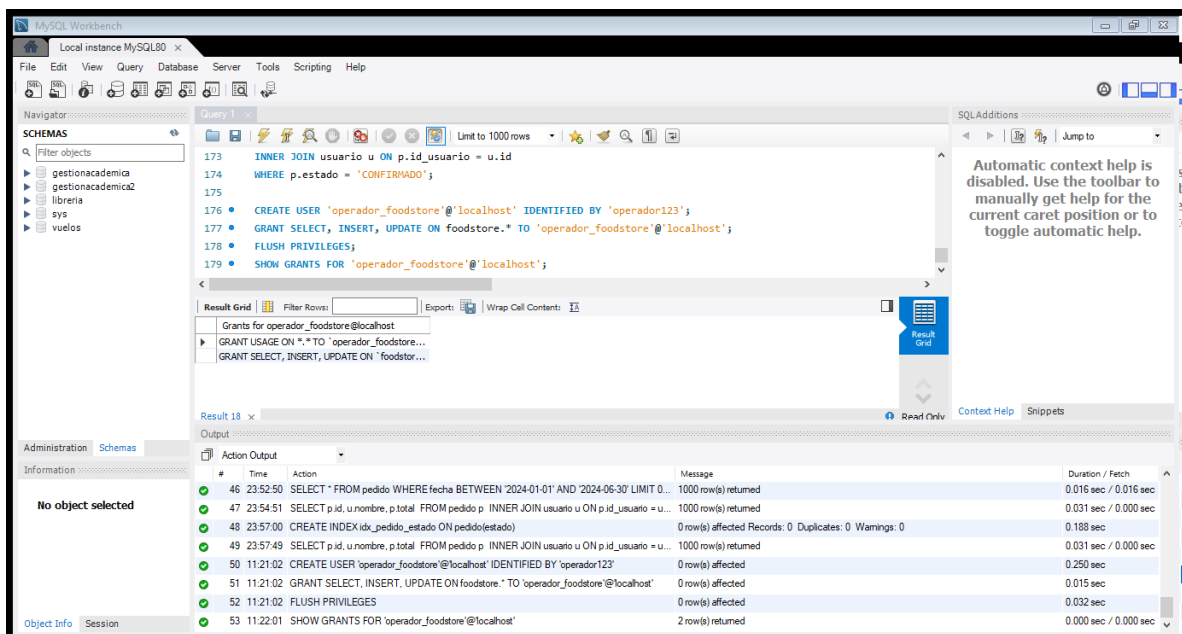
Índice en estado:



Al comparar los tiempos de ejecución, se observa que el índice en `id_usuario` mejoró notablemente el tiempo de búsqueda por igualdad (de 0.031 sec a 0.000 sec). En las consultas de rango y JOIN, MySQL ya optimizaba la búsqueda internamente, por lo que el índice no generó una diferencia significativa en este volumen de datos.

Etapas 4 – Seguridad

Captura del usuario creado con privilegios + captura de permisos (SHOW GRANTS):



Vista 1- Vista a usuarios seguros (oculta contraseña y celular).

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying a SQL script. The 'Output' tab at the bottom shows the execution results of the script.

```

171 • SELECT p.id, u.nombre, p.total
172 • FROM pedido p
173 • INNER JOIN usuario u ON p.id_usuario = u.id
174 • WHERE p.estado = 'CONFIRMADO';
175
176 • CREATE USER 'operador_foodstore'@'localhost' IDENTIFIED BY 'operador123';
177 • GRANT SELECT, INSERT, UPDATE ON foodstore.* TO 'operador_foodstore'@'localhost';
178 • FLUSH PRIVILEGES;
179 • SHOW GRANTS FOR 'operador_foodstore'@'localhost';
180
181 • CREATE VIEW vista_usuarios_seguros AS
182 • SELECT id, nombre, apellido, mail, rol
183 • FROM usuario
184 • WHERE eliminado = FALSE;

```

The 'Output' tab shows the following results:

#	Time	Action	Message	Duration / Fetch
47	23:54:51	SELECT p.id, u.nombre, p.total FROM pedido p INNER JOIN usuario u ON p.id_usuario = u...	1000 row(s) returned	0.031 sec / 0.000 sec
48	23:57:00	CREATE INDEX idx_pedido_estado ON pedido(estado)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.188 sec
49	23:57:49	SELECT p.id, u.nombre, p.total FROM pedido p INNER JOIN usuario u ON p.id_usuario = u...	1000 row(s) returned	0.031 sec / 0.000 sec
50	11:21:02	CREATE USER 'operador_foodstore'@'localhost' IDENTIFIED BY 'operador123'	0 row(s) affected	0.250 sec
51	11:21:02	GRANT SELECT, INSERT, UPDATE ON foodstore.* TO 'operador_foodstore'@'localhost'	0 row(s) affected	0.015 sec
52	11:21:02	FLUSH PRIVILEGES	0 row(s) affected	0.032 sec
53	11:22:01	SHOW GRANTS FOR 'operador_foodstore'@'localhost'	2 row(s) returned	0.000 sec / 0.000 sec
54	11:23:30	CREATE VIEW vista_usuarios_seguros AS SELECT id, nombre, apellido, mail, rol FROM usu...	0 row(s) affected	0.062 sec

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying a SQL script. The 'Result Grid' tab at the bottom shows the data returned by the view.

```

179 • SHOW GRANTS FOR 'operador_foodstore'@'localhost';
180
181 • CREATE VIEW vista_usuarios_seguros AS
182 • SELECT id, nombre, apellido, mail, rol
183 • FROM usuario
184 • WHERE eliminado = FALSE;
185 • SELECT * FROM vista_usuarios_seguros;

```

The 'Result Grid' shows the following data:

id	nombre	apellido	mail	rol
1	Victoria	Martinez	victoria@foodstore.com	ADMIN
2	Renzo	Martinez	renzo@foodstore.com	USUARIO
3	Gloria	Martinez	gloria@foodstore.com	USUARIO
4	Luis	Capello	luis@foodstore.com	USUARIO
5	Ana	Rodriguez	ana@foodstore.com	USUARIO

The 'Output' tab shows the following results:

#	Time	Action	Message	Duration / Fetch
48	23:57:00	CREATE INDEX idx_pedido_estado ON pedido(estado)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.188 sec
49	23:57:49	SELECT p.id, u.nombre, p.total FROM pedido p INNER JOIN usuario u ON p.id_usuario = u...	1000 row(s) returned	0.031 sec / 0.000 sec
50	11:21:02	CREATE USER 'operador_foodstore'@'localhost' IDENTIFIED BY 'operador123'	0 row(s) affected	0.250 sec
51	11:21:02	GRANT SELECT, INSERT, UPDATE ON foodstore.* TO 'operador_foodstore'@'localhost'	0 row(s) affected	0.015 sec
52	11:21:02	FLUSH PRIVILEGES	0 row(s) affected	0.032 sec
53	11:22:01	SHOW GRANTS FOR 'operador_foodstore'@'localhost'	2 row(s) returned	0.000 sec / 0.000 sec
54	11:23:30	CREATE VIEW vista_usuarios_seguros AS SELECT id, nombre, apellido, mail, rol FROM usu...	0 row(s) affected	0.062 sec
55	11:23:54	SELECT * FROM vista_usuarios_seguros LIMIT 0, 1000	5 row(s) returned	0.016 sec / 0.000 sec

Vista 2 – Ocultar información sensible de pedidos.

MySQL Workbench - Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

Schemas

Filter objects

gestionacademica
gestionacademica2
libreria
sys
vuelos

Query 1

```
178 • FLUSH PRIVILEGES;  
179 • SHOW GRANTS FOR 'operador_foodstore'@'localhost';  
180  
181 • CREATE VIEW vista_usuarios_seguros AS  
182 SELECT id, nombre, apellido, mail, rol  
183 FROM usuario  
184 WHERE eliminado = FALSE;  
185 • SELECT * FROM vista_usuarios_seguros;  
186  
187 • CREATE VIEW vista_pedidos_publicos AS  
188 SELECT p.id, u.nombre, u.apellido, p.fecha, p.estado, p.total  
189 FROM pedido p  
190 INNER JOIN usuario u ON p.id_usuario = u.id  
191 WHERE p.eliminado = FALSE;
```

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
49	23:57:49	SELECT p.id, u.nombre, p.total FROM pedido p INNER JOIN usuario u ON p.id_usuario = u...	1000 row(s) returned	0.031 sec / 0.000 sec
50	11:21:02	CREATE USER 'operador_foodstore'@'localhost' IDENTIFIED BY 'operador123'	0 row(s) affected	0.250 sec
51	11:21:02	GRANT SELECT, INSERT, UPDATE ON foodstore.* TO 'operador_foodstore'@'localhost'	0 row(s) affected	0.015 sec
52	11:21:02	FLUSH PRIVILEGES	0 row(s) affected	0.032 sec
53	11:22:01	SHOW GRANTS FOR 'operador_foodstore'@'localhost'	2 row(s) returned	0.000 sec / 0.000 sec
54	11:23:30	CREATE VIEW vista_usuarios_seguros AS SELECT id, nombre, apellido, mail, rol FROM usu...	0 row(s) affected	0.062 sec
55	11:23:54	SELECT * FROM vista_usuarios_seguros LIMIT 0, 1000	5 row(s) returned	0.016 sec / 0.000 sec
56	11:29:50	CREATE VIEW vista_pedidos_publicos AS SELECT p.id, u.nombre, u.apellido, p.fecha, p.es...	0 row(s) affected	0.188 sec

Administration Schemas

Information

No object selected

Object Info Session

MySQL Workbench - Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

Schemas

Filter objects

gestionacademica
gestionacademica2
libreria
sys
vuelos

Query 1

```
186  
187 • CREATE VIEW vista_pedidos_publicos AS  
188 SELECT p.id, u.nombre, u.apellido, p.fecha, p.estado, p.total  
189 FROM pedido p  
190 INNER JOIN usuario u ON p.id_usuario = u.id  
191 WHERE p.eliminado = FALSE;  
192 • SELECT * FROM vista_pedidos_publicos LIMIT 10;
```

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

Result Grid

id	nombre	apellido	fecha	estado	total
26	Victoria	Martinez	2024-05-22	CANCELADO	0.00
32	Victoria	Martinez	2024-08-04	CANCELADO	0.00
39	Victoria	Martinez	2024-09-24	TERMINADO	0.00
40	Victoria	Martinez	2024-10-12	CONFIRMADO	0.00
52	Victoria	Martinez	2024-08-13	PENDIENTE	0.00

vista_pedidos_publicos 20 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
50	11:21:02	CREATE USER 'operador_foodstore'@'localhost' IDENTIFIED BY 'operador123'	0 row(s) affected	0.250 sec
51	11:21:02	GRANT SELECT, INSERT, UPDATE ON foodstore.* TO 'operador_foodstore'@'localhost'	0 row(s) affected	0.015 sec
52	11:21:02	FLUSH PRIVILEGES	0 row(s) affected	0.032 sec
53	11:22:01	SHOW GRANTS FOR 'operador_foodstore'@'localhost'	2 row(s) returned	0.000 sec / 0.000 sec
54	11:23:30	CREATE VIEW vista_usuarios_seguros AS SELECT id, nombre, apellido, mail, rol FROM usu...	0 row(s) affected	0.062 sec
55	11:23:54	SELECT * FROM vista_usuarios_seguros LIMIT 0, 1000	5 row(s) returned	0.016 sec / 0.000 sec
56	11:29:50	CREATE VIEW vista_pedidos_publicos AS SELECT p.id, u.nombre, u.apellido, p.fecha, p.es...	0 row(s) affected	0.188 sec
57	11:30:17	SELECT * FROM vista_pedidos_publicos LIMIT 10	10 row(s) returned	0.063 sec / 0.000 sec

Administration Schemas

Information

No object selected

Object Info Session

Prueba de integridad – violación UNIQUE (mail duplicado).

```
194 • INSERT INTO usuario (nombre, apellido, mail, celular, contrasena, rol)
195     VALUES ('Test', 'Test', 'victoria@foodstore.com', '123', '123', 'USUARIO');
196
197
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
51	11:21:02	GRANT SELECT, INSERT, UPDATE ON foodstore.* TO 'operador_foodstore'@'localhost'	0 row(s) affected	0.015 sec
52	11:21:02	FLUSH PRIVILEGES	0 row(s) affected	0.032 sec
53	11:22:01	SHOW GRANTS FOR 'operador_foodstore'@'localhost'	2 row(s) returned	0.000 sec / 0.000 sec
54	11:23:30	CREATE VIEW vista_usuarios_seguros AS SELECT id, nombre, apellido, mail, rol FROM usu...	0 row(s) affected	0.062 sec
55	11:23:54	SELECT * FROM vista_usuarios_seguros LIMIT 0, 1000	5 row(s) returned	0.016 sec / 0.000 sec
56	11:29:50	CREATE VIEW vista_pedidos_publicos AS SELECT p.id, u.nombre, u.apellido, p.fecha, p.es...	0 row(s) affected	0.188 sec
57	11:30:17	SELECT * FROM vista_pedidos_publicos LIMIT 10	10 row(s) returned	0.063 sec / 0.000 sec
58	12:19:59	INSERT INTO usuario (nombre, apellido, mail, celular, contrasena, rol) VALUES ('Test', 'Test...	Error Code: 1062. Duplicate entry 'victoria@foodstore.com' for key 'usuario mail'	0.157 sec

Prueba de integridad – violación CHECK (precio negativo).

```
197 • INSERT INTO producto (nombre, precio, stock, id_categoria)
198     VALUES ('Test', -500, 10, 1);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
52	11:21:02	FLUSH PRIVILEGES	0 row(s) affected	0.032 sec
53	11:22:01	SHOW GRANTS FOR 'operador_foodstore'@'localhost'	2 row(s) returned	0.000 sec / 0.000 sec
54	11:23:30	CREATE VIEW vista_usuarios_seguros AS SELECT id, nombre, apellido, mail, rol FROM usu...	0 row(s) affected	0.062 sec
55	11:23:54	SELECT * FROM vista_usuarios_seguros LIMIT 0, 1000	5 row(s) returned	0.016 sec / 0.000 sec
56	11:29:50	CREATE VIEW vista_pedidos_publicos AS SELECT p.id, u.nombre, u.apellido, p.fecha, p.es...	0 row(s) affected	0.188 sec
57	11:30:17	SELECT * FROM vista_pedidos_publicos LIMIT 10	10 row(s) returned	0.063 sec / 0.000 sec
58	12:19:59	INSERT INTO usuario (nombre, apellido, mail, celular, contrasena, rol) VALUES ('Test', 'Test...	Error Code: 1062. Duplicate entry 'victoria@foodstore.com' for key 'usuario mail'	0.157 sec
59	12:22:44	INSERT INTO producto (nombre, precio, stock, id_categoria) VALUES ('Test', -500, 10, 1)	Error Code: 3819. Check constraint 'producto_chk_1' is violated.	0.031 sec

Procedimiento almacenado seguro (anti SQL injection).

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

- gestionacademia
- gestionacademica2
- libreria
- sys
- vuelos

Query 1

```
203 SELECT p.id, p.fecha, p.estado, p.total, p.forma_pago
204 FROM pedido p
205 WHERE p.id_usuario = p.id_usuario
206 AND p.eliminado = FALSE;
207 END //
208 DELIMITER ;
209 CALL buscar_pedidos_por_usuario(1);
```

Result Grid

id	fecha	estado	total	forma_pago
32	2024-08-04	CANCELADO	0.00	TRANSFERENCIA
39	2024-09-24	TERMINADO	0.00	TARJETA
40	2024-10-12	CONFIRMADO	0.00	TRANSFERENCIA
52	2024-08-13	PENDIENTE	0.00	TARJETA
54	2024-08-02	PENDIENTE	0.00	TRANSFERENCIA

Result 21

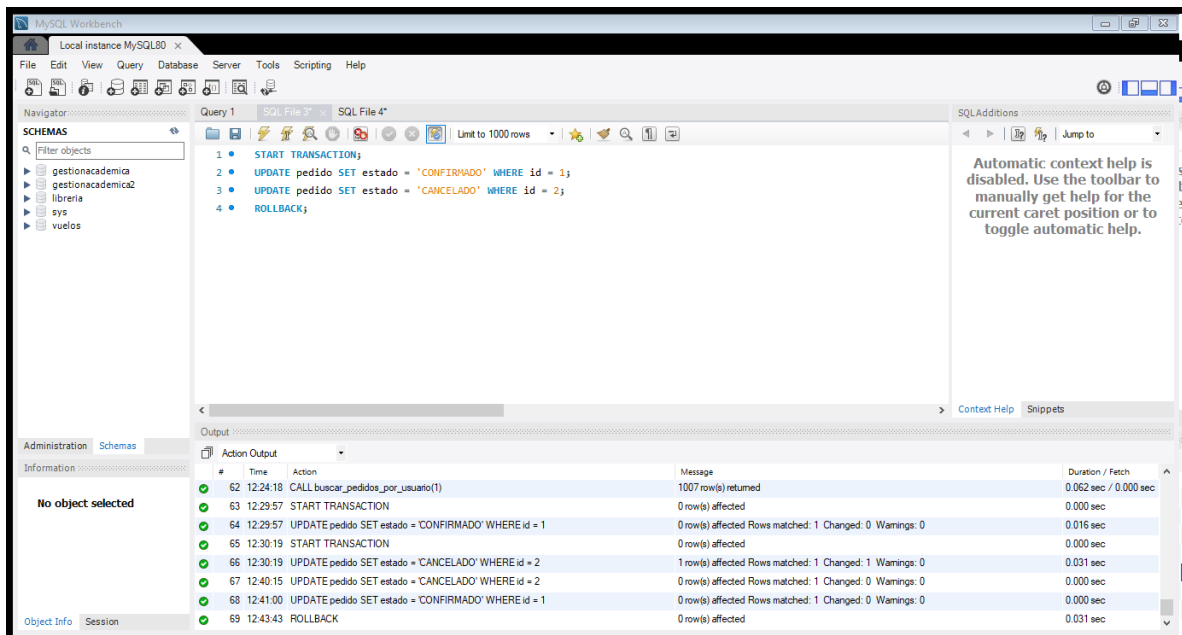
Output

Action Output

#	Time	Action	Message	Duration / Fetch
55	11:23:54	SELECT * FROM vista_usuarios_seguros LIMIT 0, 1000	5 row(s) returned	0.016 sec / 0.000 sec
56	11:29:50	CREATE VIEW vista_pedidos_publicos AS SELECT p.id, u.nombre, u.apellido, p.fecha, p.es...	0 row(s) affected	0.188 sec
57	11:30:17	SELECT * FROM vista_pedidos_publicos LIMIT 10	10 row(s) returned	0.063 sec / 0.000 sec
58	12:19:59	INSERT INTO usuario (nombre, apellido, mail, celular, contrasena, rol) VALUES ('Test', 'Test...	Error Code: 1062. Duplicate entry 'victoria@foodstore.com' for key 'usuario mail'	0.157 sec
59	12:22:44	INSERT INTO producto (nombre, precio, stock, id_categoria) VALUES ('Test', -500, 10, 1)	Error Code: 3819. Check constraint 'producto_chk_1' is violated.	0.031 sec
60	12:23:42	CREATE PROCEDURE buscar_pedidos_por_usuario(IN p_id_usuario BIGINT) BEGIN SEL...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds...	0.016 sec
61	12:23:52	CREATE PROCEDURE buscar_pedidos_por_usuario(IN p_id_usuario BIGINT) BEGIN SEL...	0 row(s) affected	0.172 sec
62	12:24:18	CALL buscar_pedidos_por_usuario(1)	1007 row(s) returned	0.062 sec / 0.000 sec

Etapas 5 – Concurrency

Captura: Simulación de deadlock (SQL file 3 y SQL file 4).



MySQL Workbench - Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

gestionacademica
gestionacademica2
libreria
sys
vuelos

Administration Schemas

Information

No object selected

Object Info Session

Query 1 SQL File 3* SQL File 4*

Limit to 1000 rows

1 • START TRANSACTION;
2 • UPDATE pedido SET estado = 'CONFIRMADO' WHERE id = 1;
3 • UPDATE pedido SET estado = 'CANCELADO' WHERE id = 2;
4 • ROLLBACK;

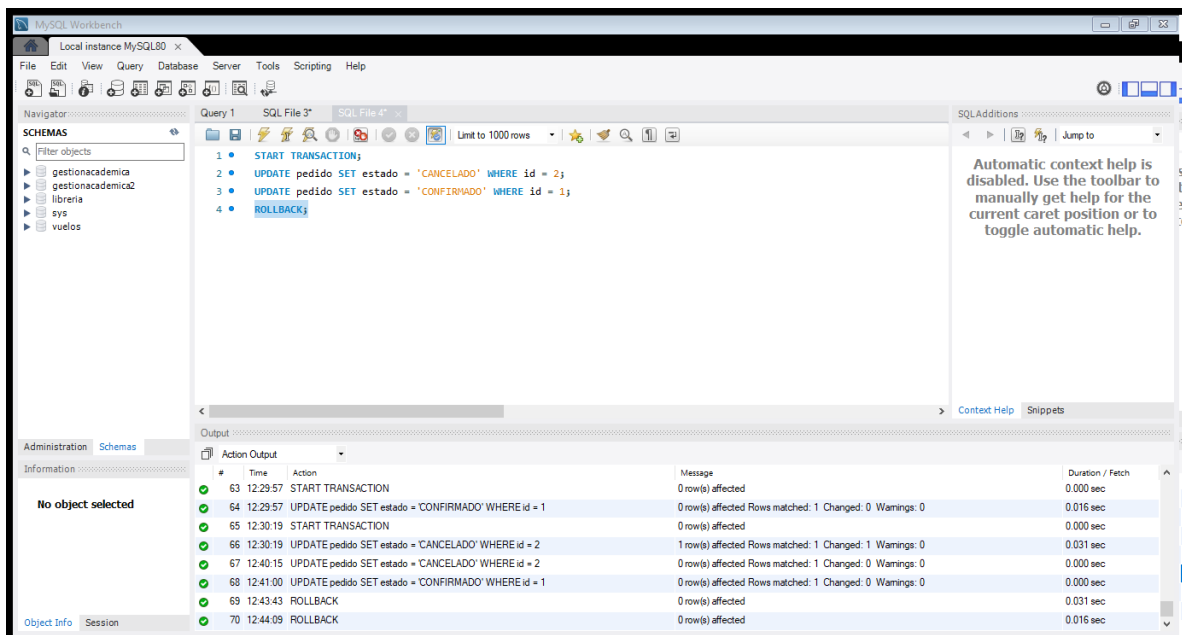
SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Output

Action Output

#	Time	Action	Message	Duration / Fetch
62	12:24:18	CALL buscar_pedidos_por_usuario(1)	1007 row(s) returned	0.062 sec / 0.000 sec
63	12:29:57	START TRANSACTION	0 row(s) affected	0.000 sec
64	12:29:57	UPDATE pedido SET estado = 'CONFIRMADO' WHERE id = 1	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0	0.016 sec
65	12:30:19	START TRANSACTION	0 row(s) affected	0.000 sec
66	12:30:19	UPDATE pedido SET estado = 'CANCELADO' WHERE id = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.031 sec
67	12:40:15	UPDATE pedido SET estado = 'CANCELADO' WHERE id = 2	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0	0.000 sec
68	12:41:00	UPDATE pedido SET estado = 'CONFIRMADO' WHERE id = 1	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0	0.000 sec
69	12:43:43	ROLLBACK	0 row(s) affected	0.031 sec



MySQL Workbench - Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

gestionacademica
gestionacademica2
libreria
sys
vuelos

Administration Schemas

Information

No object selected

Object Info Session

Query 1 SQL File 3* SQL File 4*

Limit to 1000 rows

1 • START TRANSACTION;
2 • UPDATE pedido SET estado = 'CANCELADO' WHERE id = 2;
3 • UPDATE pedido SET estado = 'CONFIRMADO' WHERE id = 1;
4 • ROLLBACK;

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Output

Action Output

#	Time	Action	Message	Duration / Fetch
63	12:29:57	START TRANSACTION	0 row(s) affected	0.000 sec
64	12:29:57	UPDATE pedido SET estado = 'CONFIRMADO' WHERE id = 1	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0	0.016 sec
65	12:30:19	START TRANSACTION	0 row(s) affected	0.000 sec
66	12:30:19	UPDATE pedido SET estado = 'CANCELADO' WHERE id = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.031 sec
67	12:40:15	UPDATE pedido SET estado = 'CANCELADO' WHERE id = 2	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0	0.000 sec
68	12:41:00	UPDATE pedido SET estado = 'CONFIRMADO' WHERE id = 1	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0	0.000 sec
69	12:43:43	ROLLBACK	0 row(s) affected	0.031 sec
70	12:44:09	ROLLBACK	0 row(s) affected	0.016 sec

Transacciones con COMMIT y ROLLBACK.

The screenshot shows the MySQL Workbench interface with a query editor containing a transaction script. The script consists of two parts: the first part updates the state of a record with id=1 to 'CONFIRMADO' and id=2 to 'CANCELADO', followed by a ROLLBACK; the second part starts a new transaction, updates the state of a record with id=3 to 'CONFIRMADO' and its total to 5000, followed by a COMMIT.

```
1 • START TRANSACTION;
2 • UPDATE pedido SET estado = 'CONFIRMADO' WHERE id = 1;
3 • UPDATE pedido SET estado = 'CANCELADO' WHERE id = 2;
4 • ROLLBACK;
5
6 • START TRANSACTION;
7 • UPDATE pedido SET estado = 'CONFIRMADO' WHERE id = 3;
8 • UPDATE pedido SET total = 5000 WHERE id = 3;
9 • COMMIT;
```

The Output window shows the execution results:

#	Time	Action	Message	Duration / Fetch
67	12:40:15	UPDATE pedido SET estado = 'CANCELADO' WHERE id = 2	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0	0.000 sec
68	12:41:00	UPDATE pedido SET estado = 'CONFIRMADO' WHERE id = 1	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0	0.000 sec
69	12:43:43	ROLLBACK	0 row(s) affected	0.031 sec
70	12:44:09	ROLLBACK	0 row(s) affected	0.016 sec
71	12:46:07	START TRANSACTION	0 row(s) affected	0.000 sec
72	12:46:07	UPDATE pedido SET estado = 'CONFIRMADO' WHERE id = 3	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0	0.000 sec
73	12:46:08	UPDATE pedido SET total = 5000 WHERE id = 3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
74	12:46:08	COMMIT	0 row(s) affected	0.062 sec

The screenshot shows the MySQL Workbench interface with a query editor containing a transaction script. The script starts with a transaction, updates the state of a record with id=3 to 'CONFIRMADO' and its total to 5000, followed by a COMMIT. Then it starts another transaction, updates the state of a record with id=3 to 'CANCELADO', followed by a ROLLBACK. Finally, it executes a SELECT query to retrieve the details of the record with id=3.

```
7 • UPDATE pedido SET estado = 'CONFIRMADO' WHERE id = 3;
8 • UPDATE pedido SET total = 5000 WHERE id = 3;
9 • COMMIT;
10 • START TRANSACTION;
11 • UPDATE pedido SET estado = 'CANCELADO' WHERE id = 3;
12 • ROLLBACK;
13 • SELECT id, estado, total FROM pedido WHERE id = 3;
```

The Result Grid shows the output of the SELECT query:

id	estado	total
3	CONFIRMADO	5000.00

The Output window shows the execution results:

#	Time	Action	Message	Duration / Fetch
71	12:46:07	START TRANSACTION	0 row(s) affected	0.000 sec
72	12:46:07	UPDATE pedido SET estado = 'CONFIRMADO' WHERE id = 3	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0	0.000 sec
73	12:46:08	UPDATE pedido SET total = 5000 WHERE id = 3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
74	12:46:08	COMMIT	0 row(s) affected	0.062 sec
75	12:47:43	START TRANSACTION	0 row(s) affected	0.016 sec
76	12:47:43	UPDATE pedido SET estado = 'CANCELADO' WHERE id = 3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
77	12:47:43	ROLLBACK	0 row(s) affected	0.015 sec
78	12:47:56	SELECT id, estado, total FROM pedido WHERE id = 3 LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

READ COMMITTED.

The screenshot shows the MySQL Workbench interface with a query window containing the following SQL script:

```
11 • UPDATE pedido SET estado = 'CANCELADO' WHERE id = 3;
12 • ROLLBACK;
13
14 • SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
15 • START TRANSACTION;
16 • SELECT estado FROM pedido WHERE id = 3;
17 • COMMIT;
```

The result grid shows the state of the 'pedido' table, with the 'estado' column containing the value 'CONFIRMADO'.

The output window displays the following actions:

#	Time	Action	Message	Duration / Fetch
75	12:47:43	START TRANSACTION	0 row(s) affected	0.016 sec
76	12:47:43	UPDATE pedido SET estado = 'CANCELADO' WHERE id = 3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
77	12:47:43	ROLLBACK	0 row(s) affected	0.015 sec
78	12:47:56	SELECT id, estado, total FROM pedido WHERE id = 3 LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
79	12:52:50	SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED	0 row(s) affected	0.031 sec
80	12:52:50	START TRANSACTION	0 row(s) affected	0.016 sec
81	12:52:50	SELECT estado FROM pedido WHERE id = 3 LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
82	12:52:50	COMMIT	0 row(s) affected	0.000 sec

REPEATABLE READ.

The screenshot shows the MySQL Workbench interface with a query window containing the following SQL script:

```
15 • START TRANSACTION;
16 • SELECT estado FROM pedido WHERE id = 3;
17 • COMMIT;
18 • SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
19 • START TRANSACTION;
20 • SELECT estado FROM pedido WHERE id = 3;
21 • COMMIT;
```

The result grid shows the state of the 'pedido' table, with the 'estado' column containing the value 'CONFIRMADO'.

The output window displays the following actions:

#	Time	Action	Message	Duration / Fetch
79	12:52:50	SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED	0 row(s) affected	0.031 sec
80	12:52:50	START TRANSACTION	0 row(s) affected	0.016 sec
81	12:52:50	SELECT estado FROM pedido WHERE id = 3 LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
82	12:52:50	COMMIT	0 row(s) affected	0.000 sec
83	12:54:38	SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ	0 row(s) affected	0.000 sec
84	12:54:38	START TRANSACTION	0 row(s) affected	0.000 sec
85	12:54:38	SELECT estado FROM pedido WHERE id = 3 LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
86	12:54:38	COMMIT	0 row(s) affected	0.000 sec

Referencias a Scripts SQL

Los scripts SQL del proyecto están organizados en el siguiente orden de ejecución:

- 01_esquema.sql → Creación de tablas y constraints.
- 02_catalogos.sql → Datos base (categorías, usuarios, productos).
- 03_carga_masiva.sql → Generación de 15000+ registros.
- 04_indices.sql → Creación de índices.
- 05_consultas.sql → Consultas avanzadas (JOIN, GROUP BY, subconsulta).
- 06_vistas.sql → Vistas de seguridad.
- 07_seguridad.sql → Usuario, permisos y procedimiento almacenado.
- 08_transacciones.sql → Transacciones con COMMIT y ROLLBACK.
- 09_concurrencia_guiada.sql → Niveles de aislamiento y concurrencia.

Anexo IA – Evidencia de uso de Inteligencia Artificial

Herramienta utilizada: Claude (Anthropic)

Etapas 1 - Modelado y Constraints

Consulta: Le consulté a Claude sobre cómo crear las tablas con sus constraints (PK, FK, UNIQUE, CHECK) en MySQL Workbench para el sistema Food Store.

Respuesta de Claude: Me guió en la creación de cada tabla en orden correcto respetando las foreign keys, empezando por `categoria`, luego `producto`, `usuario`, `pedido` y finalmente `detalle_pedido`. Me explicó que debía crear primero las tablas referenciadas antes que las que tienen FK.

Etapas 2 – Carga Masiva

Consulta: le pregunté cómo generar más de 10000 registros usando SQL puto sin insertar uno por uno.

Respuesta de Claude: me sugirió usar un producto cartesiano con `information_schema.columns` para generar 5000 pedidos y 10000 detalles de pedido usando `RAND()` y `ELT()` para valores aleatorios.

Etapas 3 – Consultas Avanzadas

Consulta: le pregunté cómo medir los tiempos de ejecución con y sin índice en Workbench.

Respuesta de Claude: me explicó que el tiempo aparece bajo a la derecha en Workbench después de ejecutar cada consulta, y me guió para crear los índices con `CREATE INDEX` y comparar los tiempos antes y después.

Etapas 4 – Seguridad

Consulta: Le consulté cómo crear un usuario con privilegios mínimos en MySQL y cómo hacer un procedimiento almacenado seguro.

Respuesta de Claude: Me guió para crear el usuario con `CREATE USER` y asignarle solo permisos de `SELECT`, `INSERT` y `UPDATE` con `GRANT`. También me explicó que un procedimiento almacenado evita SQL injection porque los parámetros se tratan como datos y no como código SQL.

Etapas 5 – Concurrencia y Transacciones

Consulta: Intenté simular un deadlock en dos sesiones de MySQL Workbench actualizando las mismas filas en orden inverso, pero MySQL resolvió el conflicto automáticamente y ambas transacciones completaron con éxito sin generar el error 1213.

Respuesta de Claude: Esto es normal en MySQL/InnoDB. El motor detecta automáticamente los ciclos de espera y elige una transacción como víctima cancelándola con el error ERROR 1213 (40001): Deadlock found. En bases de datos con mayor volumen de transacciones concurrentes simultáneas el deadlock es más fácil de reproducir. Lo que observaste demuestra que InnoDB tiene mecanismos de detección y resolución automática de deadlocks.

Bibliografía

Material de cátedra — Unidad 5:

- Material de cátedra. (2026). *Apunte: Vistas en Bases de Datos*. Tecnicatura Universitaria en Programación a Distancia, UTN.
- Material de cátedra. (2026). *Bases de Datos I — Módulo 5: Guía Práctica*. Tecnicatura Universitaria en Programación a Distancia, UTN.
- Material de cátedra. (2026). *Fundamentos de SQL: Lenguaje para Bases de Datos Relacionales*. Tecnicatura Universitaria en Programación a Distancia, UTN.
- Material de cátedra. (2026). *Módulo 5: Subconsultas*. Tecnicatura Universitaria en Programación a Distancia, UTN.

Material de cátedra — Unidad 6:

- Material de cátedra. (2026). *Bases de Datos DDL*. Tecnicatura Universitaria en Programación a Distancia, UTN.
- Material de cátedra. (2026). *Sistemas de Gestión de Bases de Datos: Arquitectura e Implementación*. Tecnicatura Universitaria en Programación a Distancia, UTN.

Material de cátedra — Unidad 7:

- Material de cátedra. (2026). *Integridad y Transacciones*. Tecnicatura Universitaria en Programación a Distancia, UTN.
- Material de cátedra. (2026). *Integridad de la Base de Datos y Transacciones: Garantizar la Confiabilidad de los Datos*. Tecnicatura Universitaria en Programación a Distancia, UTN.

Material de cátedra — Unidad 8:

- Material de cátedra. (2026). *Privacidad y Seguridad en BD: Material Complementario*. Tecnicatura Universitaria en Programación a Distancia, UTN.
- Material de cátedra. (2026). *Seguridad Avanzada en Bases de Datos*. Tecnicatura Universitaria en Programación a Distancia, UTN.
- Material de cátedra. (2026). *Seguridad y Privacidad de la Base de Datos: Protección de su Activo más Valioso*. Tecnicatura Universitaria en Programación a Distancia, UTN.

Documentación oficial:

- Oracle Corporation. (s.f.). *MySQL 8.0 Reference Manual*.
<https://dev.mysql.com/doc/refman/8.0/en/>
- Oracle Corporation. (s.f.). *MySQL Workbench Manual*.
<https://dev.mysql.com/doc/workbench/en/>

Links

Repositorio GitHub: <https://github.com/vmartinez22/FoodStore-BD>

Video explicativo: <https://youtu.be/nuYb8Vz9Nk8>